Unit 5

POLYMORPHISM AND TYPE CONVERSION

Polymorphism:

Multiple functions in the same scope with the same name but different parameter lists (different number/types/order).

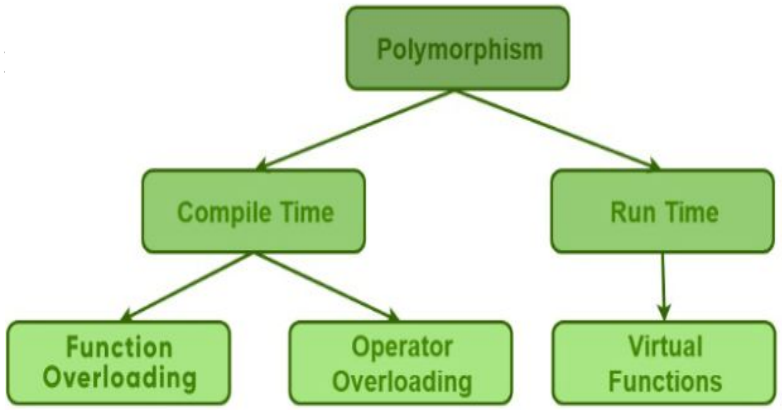
Selection: Done at **compile time** by the compiler using the function signature.

The word polymorphism means having many forms. A real-life example of polymorphism is a person who at the same time can have different characteristics. A man at the same time is a father, a husband, and an employee. So, the same person exhibits different behaviour in different situations. This is called polymorphism.

In C++, polymorphism concept can be applied to functions and operators. A single function can work differently in different situations. Similarly, an operator works different when used in different context.

Types of Polymorphism
Polymorphism in C++ can be classified into two types

1. Compile-time Polymorphism
2. Runtime Polymorphism



1. Compile-Time Polymorphism

Also known as early binding and static polymorphism, in compile-time polymorphism, the compiler determines how the function or operator will work depending on the context. This type of polymorphism is achieved by function overloading or operator overloading.

A. Function Overloading

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but behave differently for different parameters. Such functions are said to be overloaded; hence, this is known as Function Overloading. Functions can be overloaded either by changing the number of arguments or changing the type of arguments.

Example: Function Overloading

```
#include <iostream>
using namespace std;

class Math {
public:
   // 1. Add two integers
   int add(int a, int b) {
      return a + b;
   }

   // 2. Add three integers
   int add(int a, int b, int c) {
      return a + b + c;
   }

   // 3. Add two doubles
   double add(double a, double b) {
      return a + b;
   }
};
```

```
int main() {
   Math m;
   cout << "Add two ints: " << m.add(5, 10) << endl;
   cout << "Add three ints: " << m.add(5, 10, 15) << endl;
   cout << "Add two doubles: " << m.add(2.5, 3.5) << endl;
   return 0;
}
```

```
//Output
Add two ints: 15
Add three ints: 30
Add two doubles: 6
```

- **The compiler decides which add() to call based on the arguments.**

B. Operator Overloading:

In C++, operators like +, -, *, etc. can be redefined (overloaded) to work with user-defined types (objects).

Rules:

1. At least one operand must be a user-defined type (class object).
2. Cannot overload: ::, ., .*, sizeof, ?:

Types:

1. **Unary Operator Overloading:**

Unary operators operate on a single operand. Examples: ++, --, - (negation).

2. **Binary Operator Overloading:**

Binary operators work on two operands. Examples: +, -, *, /.

1. Example: Unary Operator Overloading

```
#include <iostream>
using namespace std;

class Number {
    int x;
public:
    Number(int a) : x(a) {} // constructor using initializer list


    // Overload unary minus operator
    Number operator-() {
        return Number(-x);
    }

    void display() {
        cout << "Value: " << x << endl;
    }
};
int main() {
    Number n1(10);
    Number n2 = -n1;  // Calls operator-()
    n1.display();
    n2.display();
    return 0;
```

- **Overloading happens by writing a function operator-() inside the class.**
- **When -n1 is used, it calls n1.operator-().**
- **That function returns a new object with the negated value.**

```
//Output
Value: 10
Value: -10
```

Here, -n1 internally calls n1.operator-().

2. Binary Operator Overloading:

Binary operators work on two operands. Examples: +, -, *, /.

Example: Binary Operator Overloading

```
#include <iostream>
using namespace std;

class Complex {
   int real, imag;
public:
   Complex(int r = 0, int i = 0) {
      real = r;
      imag = i;
   }

   // Overload + operator
   Complex operator+(Complex c) {
      return Complex(real + c.real, imag + c.imag);
   }

   void display() {
      cout << real << " + " << imag << "i" << endl;
   }
};
```

```
int main() {
   Complex c1(3, 4), c2(1, 2);
   Complex c3 = c1 + c2;  // Calls operator+()
   c1.display();
   c2.display();
   cout << "After Addition: ";
   c3.display();
   return 0;
}
```

```
//Output
3 + 4i
1 + 2i
After Addition: 4 + 6i
```

Here, c1 + c2 calls c1.operator+(c2).

- Function Overloading → Same function name, different signatures.

- Operator Overloading → Redefine operators for objects.

1. Unary → One operand (e.g., -obj, ++obj).
2. Binary → Two operands (e.g., obj1 + obj2).

- Compiler decides at compile time which version to use → Compile Time Polymorphism.

Function Overloading	Function Overriding
Function Overloading provides multiple definitions of the function by changing signature.	Function Overriding is the redefinition of base class function in its derived class with same signature.
An example of compile time polymorphism.	An example of run time polymorphism.
Function signatures should be different.	Function signatures should be the same.
Overloaded functions are in same scope.	Overridden functions are in different scopes.
Overloading is used when the same function has to behave differently depending upon parameters passed to them.	Overriding is needed when derived class function has to do some different job than the base class function.
A function has the ability to load multiple times.	A function can be overridden only a single time.
In function overloading, we don't need inheritance.	In function overriding, we need an inheritance concept.

2. Run Time Polymorphism:

- Polymorphism means "many forms."
- Runtime polymorphism happens when the decision of which function to call is made at runtime (not compile time).
- Also known as late binding and dynamic polymorphism, the function call in Runtime polymorphism is resolved at runtime in contrast with compile time polymorphism, where the compiler determines which function call to bind at compilation. Runtime polymorphism is implemented using function overriding with virtual functions.
- Achieved mainly using virtual functions and base class pointers/references.

Function Overriding:

Function Overriding occurs when a derived class defines one or more member functions of the base class. That base function is said to be overridden. The base class function must be declared as virtual function for runtime polymorphism to happen.

```
#include <iostream>
using namespace std;

class Base {
public:

    // Virtual function
    virtual void display() {
        cout << "Base class function";
    }
};

class Derived : public Base {
public:

    // Overriding the base class function
    void display() override {
        cout << "Derived class function";
    }
};

int main() {

    // Creating a pointer of type Base
    Base* basePtr;

    // Creating an object of Derived class
    Derived derivedObj;

    // Pointing base class pointer to
    // derived class object
    basePtr = &derivedObj;

    // Calling the display function
    // using base class pointer
    basePtr->display();
    return 0;
}

//output
Derived class function
```

Compile Time Polymorphism	Run time Polymorphism
Also called static binding	Also called dynamic binding
Achieved using function overloading and operator overloading	Achieved using **virtual functions** and **function overriding**
Decision made by the compiler at compile time	Decision made at **runtime using vtables**
Faster due to early binding	**More flexible** but slightly slower

Pointers in C++:

A pointer stores the address of a variable.

Example:

```
int a = 10;
int* p = &a;   // pointer to int
cout << *p;    // dereference, prints 10
```

Pointers to Objects:
Just like primitive types, you can have pointers to objects.

Example:

```
#include <iostream>
using namespace std;

class Student {
    int roll;
public:
    Student(int r) : roll(r) {}
    void show() { cout << "Roll No = " << roll << endl; }
};

int main() {
    Student s1(101);
    Student* ptr = &s1;   // pointer to object
    ptr->show();          // use -> to call member function
    return 0;
}

// Output:
Roll No = 101
```

The *this* Pointer:

- Every object has a hidden pointer called this that points to the current object.
- Useful when:
  Parameter names are same as data members.
  Returning the current object.

Example:

```
#include <iostream>
using namespace std;

class Student {
   int roll;
public:
   Student(int roll) { this->roll = roll; } // disambiguation
   void show() { cout << "Roll No = " << this->roll << endl; }
};

int main() {
   Student s(101);
   s.show();
   return 0;
}
// Output:
Roll No = 101
```

Pointers to Derived Classes:
A base class pointer can point to a derived class object.
But:
Without virtual, it calls base function (static binding).
With virtual, it calls derived function (dynamic binding).

Example:

```
#include <iostream>
using namespace std;

class Base {
public:
    void show() { cout << "Base function" <<
endl; }
};

class Derived : public Base {
public:
    void show() { cout << "Derived function"
<< endl; }
};

int main() {
    Base* ptr;
    Derived d;
    ptr = &d;
    ptr->show();   // Calls Base function
(no virtual, static binding)
    return 0;
}


// Output:

Base function
```

Virtual Functions:

Declared with keyword virtual in base class.

Enables dynamic binding: function call resolved at runtime.

Example:

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual void show() { cout << "Base function" << endl; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived function" << endl; }
};
int main() {
    Base* ptr;
    Derived d;
    ptr = &d;
    ptr->show();   // Calls Derived function (runtime polymorphism)
    return 0;
}
//Output:
Derived function
```

Static vs Dynamic Binding:

Static Binding (Early Binding)	**Dynamic Binding (Late Binding)**
Function call decided at compile time.	Function call decided at runtime.
Happens for normal functions.	Happens for virtual functions.
Faster, but no flexibility.	Enables polymorphism.

Pointer to object → allows indirect access.

this pointer → refers to current object.

Base pointer → Derived object:

Without virtual: base function executes (static binding).

With virtual: derived function executes (dynamic binding).

Virtual function → main tool for runtime polymorphism.

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void display() { cout << "Base class display()" << endl; }
};

class Derived : public Base {
public:
    void display() override { cout << "Derived class display()" << endl; }
};

int main() {
    Base* bptr;           // Base class pointer
    Derived d;
    bptr = &d;            // Base pointer points to derived object

    bptr->display();      // Calls Derived function (dynamic binding)
    return 0;
}

//output
Derived class display()
```

Type Conversion :

Type conversion means converting one type of data to another compatible type such that it doesn't lose its meaning. It is essential for managing different data types in C++.

In C++, it happens in two ways:

1.Implicit Conversion (Type Casting by Compiler)

- Automatic conv ersion done by compiler.
- Example: int x = 5.5; // double 5.5 converted to int 5

2. Explicit Conversion (User-defined / Programmer-defined)

Conversion between:

- Basic type ↔ Class type
- Class type ↔ Basic type
- One class type ↔ Another class type

:**Implicit conversion:** compiler converts automatically (e.g., int → double).

User-defined conversions: conversions between user classes and built-in types. Use:

1. **Basic → Class**: single-argument constructor (can be implicit unless marked explicit).
2. **Class → Basic**: conversion operator operator TYPE().
3. **Class → Class**: conversion constructor in target class or operator Target() in source class.

explicit keyword prevents implicit conversions and requires direct construction.

Basic Type → Class Type :

Achieved using a constructor in the class.

Syntax:

ClassName(basic_type value);

Example:

```
#include <iostream>
using namespace std;

class Time {
    int hours, minutes;
public:
    // Constructor to convert minutes (int) → Time
    Time(int m) {
        hours = m / 60;
        minutes = m % 60;
    }
    void display() {
        cout << hours << " Hours and " <<
minutes << " Minutes" << endl;
    }
};
```

```
int main() {
    int totalMinutes = 130;
    Time t = totalMinutes;   // basic → class
    t.display();
    return 0;
}
```

//Output:

2 Hours and 10 Minutes

Class Type → Basic Type:
Achieved using a conversion function inside class.
Syntax:
operator basic_type();

Example:

```
#include <iostream>
using namespace std;

class Time {
    int hours, minutes;
public:
    Time(int h, int m) {
        hours = h;
        minutes = m;
    }

    // Conversion function: Time → int
    operator int() {
        return hours * 60 + minutes;
    }
};

int main() {
    Time t(2, 30);
    int totalMinutes = t;   // class → basic
    cout << "Total Minutes = " << totalMinutes
<< endl;
    return 0;
}
```

//Output:

Total Minutes = 150

One Class Type → Another Class Type:

Achieved in two ways:

Constructor in destination class (takes source class object).

Conversion function in source class (returns destination object).

Example 1 (Using Constructor in Destination Class)

```
#include <iostream>
using namespace std;
class Celsius {
    float temp;
public:
    Celsius(float t) : temp(t) {}
    float getTemp() { return temp; }
};
class Fahrenheit {
    float temp;
public:
    // Constructor: convert Celsius → Fahrenheit
    Fahrenheit(Celsius c) {
        temp = (c.getTemp() * 9/5) + 32;
    }
    void display() {
        cout << "Temperature in Fahrenheit: " << temp
<< endl;
    }
};
```

```
int main() {
    Celsius c(37);
    Fahrenheit f = c;   // class → class
    f.display();
    return 0;
}
```

//Output:

Temperature in Fahrenheit: 98.6

Example 2 (Using Conversion Function in Source Class)

```
#include <iostream>
using namespace std;

class Fahrenheit;  // forward declaration

class Celsius {
    float temp;
public:
    Celsius(float t) : temp(t) {}
    float getTemp() { return temp; }

    // Conversion function: Celsius → Fahrenheit
    operator Fahrenheit();
};

class Fahrenheit {
    float temp;
public:
    Fahrenheit(float t) : temp(t) {}
    void display() {
        cout << "Temperature in Fahrenheit: " <<
temp << endl;
    }
};

// Define conversion function outside
Celsius::operator Fahrenheit() {
    return Fahrenheit((temp * 9/5) + 32);
}

int main() {
    Celsius c(0);
    Fahrenheit f = c;   // Celsius → Fahrenheit
    f.display();
    return 0;
}
/
/Output:
Temperature in Fahrenheit: 32
```

Data Conversion Example (All-in-One)

```
#include <iostream>
using namespace std;

class Distance {
    int meters;
public:
    Distance(int m = 0) : meters(m) {}

    // Class → Basic
    operator int() {
        return meters;
    }

    // Basic → Class
    Distance(double km) {
        meters = km * 1000;
    }

    void display() {
        cout << "Distance = " << meters << " meters" <<
endl;
    }
};

int main() {
    // Basic → Class
    Distance d1 = 2.5;   // 2.5 km
    d1.display();

    // Class → Basic
    int m = d1;
    cout << "Distance in meters (basic type) = "
<< m << endl;

    return 0;
}


//Output:
Distance = 2500 meters
Distance in meters (basic type) = 2500
```

Basic → Class → via constructor.

Class → Basic → via conversion function (operator basic_type()).

Class → Class → via constructor in destination class OR conversion function in source class.